

Unified Project Plan: AI-Assisted Fiscal Policy Dashboard

ECO 317 – Assignment #3

Project goal in plain language. Build one Streamlit app that solves a medium-scale Smets–Wouters (2007) style DSGE model, uses QZ decomposition (Blanchard–Kahn) to produce the rational-expectations solution, simulates 1,000 periods of unconditional business-cycle dynamics, runs 40-quarter fiscal experiments with user-chosen financing rules, reports impact and cumulative multipliers with IRFs for output, consumption, investment, and debt, uses polished academic styling (dark/navy background, Garamond font), and is reliable enough to launch live with `streamlit run app.py` in front of the professor.

1. Team Roster

Role	Member
Member 1	_____
Member 2	_____
Member 3	_____
Member 4	_____
Member 5	_____
Member 6	_____

How to use this sheet. Each teammate signs their name next to the steps they own in the ownership table at the end of the document. The blanks here are the master list. Max 6 per group per the assignment.

2. Project Objective and Assignment Requirements

2.1. What the finished project must do

1. Simulate a linearized medium-scale Smets–Wouters style economy with:
 - a fraction of “rule-of-thumb” (non-Ricardian) households alongside forward-looking optimizers,
 - habit formation in consumption (h),
 - variable capital utilization (cost convexity ψ),
 - Calvo price stickiness (θ_p),
 - Calvo wage stickiness (θ_w),
 - a fiscal rule with debt-feedback aggressiveness (ϕ_b).
2. Solve the linear rational-expectations system using Blanchard–Kahn conditions or the generalized Schur (QZ) decomposition (e.g., `scipy.linalg.ordqz`).
3. Provide a unified Streamlit dashboard with a **tabbed interface**:
 - **Tab 1 – Model Fit (Unconditional Dynamics):** simulate at least 1,000 periods, report ≥ 10 business-cycle moments, dynamic f-string economic commentary.

- **Tab 2 – Fiscal Exercises (Conditional Policy):** 40-quarter IRFs for G_c shock, G_I shock, labor-tax cut, capital-tax cut; financing-rule selector; impact and cumulative multipliers; automated policy briefing.
4. Global sidebar with continuous sliders for all structural parameters above.
 5. Variable financing rules: Lump-Sum transfers, Consumption Tax Hikes, Labor Tax Hikes, Capital Tax Hikes, Government Spending Cuts.
 6. Impact multiplier and cumulative multiplier (discounted sum of output response divided by discounted sum of the shock).
 7. IRF subplots for Output, Consumption, Investment, Government Debt.
 8. “Fiscal Drag Horizon” – the exact quarter when distortionary financing forces output below steady state.
 9. Dark/navy interface with bright white chart areas and Garamond font throughout.
 10. Dynamic written summaries built from live results using Python f-strings.
 11. Be explainable by any teammate during the live demo.

2.2. Suggested project folder structure

Project 3 starts from a clean folder. Before anyone writes serious code, the group should agree on a skeleton so later steps have a target. The suggested layout below mirrors the Project 2 style and keeps UI logic separate from economics and numerics.

```
ECO 317 Project 3/
|-- app.py
|-- config.py
|-- README.md
|-- requirements.txt
|-- assets/style.css
|-- dsge/
| |-- model.py # linearized Smets-Wouters equations
| |-- steady_state.py # analytic or numerical SS solver
| |-- calibration.py # baseline parameter values
|-- solvers/
| |-- rational_expectations.py # QZ / Blanchard-Kahn
| |-- state_space.py # build A, B, C, D matrices
|-- simulation/
| |-- simulate.py # long-horizon stochastic simulation
| |-- moments.py # variances, correlations, autocorrelations
| |-- irf.py # impulse responses
|-- policy/
| |-- shocks.py #  $G_c$ ,  $G_I$ ,  $\tau_L$  cut,  $\tau_K$  cut
| |-- financing.py # lump-sum, cons tax, labor tax, etc.
| |-- multipliers.py # impact and cumulative multipliers
|-- utils/
| |-- summaries.py # f-string text generators
|-- tests/
| |-- test_steady_state.py
| |-- test_blanchard_kahn.py
| |-- test_moments.py
| |-- test_irf.py
| |-- test_multipliers.py
```

Important repo rule. Once these folders and files are created at the start of Step 3, team-mates should *edit* those files or create new files *inside those existing folders* when needed. Do not create a second top-level project layout. Keep UI in `app.py`, economics in `dsge/`, numerics in `solvers/` and `simulation/`, and policy logic in `policy/`.

2.3. How to think about the project

Simple mental model. Steady-state pins down the log-linearization point. The DSGE block lists the linearized equations. The QZ solver turns those equations into a state-space transition $x_{t+1} = Ax_t + B\varepsilon_{t+1}$ and an observation map $y_t = Cx_t$. Unconditional simulation draws random shocks and produces moments (Tab 1). IRFs turn off the random shocks and turn on one fiscal shock, then read the response off the same transition map (Tab 2). Streamlit is just the display layer.

3. Locked-In Model Specs

3.1. Shared rules for the DSGE block

Item	Locked-in choice
Model family	Medium-scale Smets–Wouters (2007) style, linearized around steady state
Households	Mixture of forward-looking optimizers and rule-of-thumb (non-Ricardian) consumers
Frictions	Habit formation, variable capital utilization, Calvo price and wage stickiness
Fiscal block	Government consumption G_c , government investment G_I , labor tax τ_L , capital tax τ_K , consumption tax τ_C , lump-sum T , debt B with rule ϕ_b
Monetary block	Locked in: Taylor rule on inflation and output gap (see §3.5 below). No “optional” language – without a nominal anchor the Phillips curves do not close and Blanchard–Kahn will fail.
Solution method	Blanchard–Kahn / generalized Schur (QZ) decomposition
Simulation target	At least 1,000 periods in Tab 1
IRF horizon	40 quarters in Tab 2
Moments	At least 10, covering variances, contemporaneous correlations with output, and first-order autocorrelations of Output, Consumption, Investment, Labor, Inflation
Multipliers	Impact multiplier and cumulative (discounted) multiplier
Runtime text	Deterministic Python f-strings driven by computed moments and IRFs
Dashboard	Single Streamlit app with global sidebar + two tabs

Non-negotiable rule. If the AI (ChatGPT, Gemini, Claude) produces equations that differ from the course notes or from the Smets–Wouters formulation the group has locked in, the course notes win. The biggest grading risk is not a missing widget. It is a misspecified Euler equation, Phillips curve, or government budget constraint.

Known AI-hallucination traps (check every generated block against this list).

- **Calvo Phillips curve.** Intercept dropped or mixed indexed/non-indexed versions. Verify the limit behavior in §3.3.
- **Euler with habit.** Shortcut marginal utility $(C_t - hC_{t-1})^{-\sigma}$ used in place of the full internal-habit FOC. We lock in the full FOC in §3.2.
- **Missing β in the Euler.** The discount factor goes on the $\mathbb{E}_t[\Lambda_{t+1}(1 + r_{t+1})]$ side, not the current-period side.
- **Capital timing.** Period- t production uses K_{t-1} , not K_t . AI output routinely silently uses K_t , which breaks the investment FOC.
- **Government budget sign errors.** Taxes shrink debt (enter with a minus sign); transfers grow debt (enter with a plus sign). Both sides of §3.4 must match the sign convention in §3.2.
- **Fiscal-rule sign for G_c -cut.** Positive ϕ_b on a G_c -cut rule is a sign error. See §3.4.
- **Steady-state ratios in multipliers.** Converting log-deviation IRFs to level multipliers requires multiplying by the steady-state Y/G ratio. Forgetting this scales the multiplier by 5–10x in either direction.
- **Taylor rule omission.** A Phillips-curve model with no nominal anchor is under-determined; see §3.5.

Every AI-generated equation or code block must be cross-checked against this list before it lands on `main`.

3.2. Household block

Forward-looking optimizers (fraction $1 - \lambda$): maximize expected utility subject to an intertemporal budget constraint with capital, bonds, consumption, and labor. The marginal utility of consumption with *internal* habits is

$$\Lambda_t^o = (C_t^o - hC_{t-1}^o)^{-\sigma} - \beta h \mathbb{E}_t[(C_{t+1}^o - hC_t^o)^{-\sigma}],$$

and the Euler equation is

$$\Lambda_t^o = \beta \mathbb{E}_t[\Lambda_{t+1}^o(1 + r_{t+1})].$$

Habit-FOC trap. A shortcut version $\lambda_t^o = (C_t^o - hC_{t-1}^o)^{-\sigma}$ (dropping the $-\beta h \mathbb{E}_t[\cdot]$ term) is common in textbook expositions but materially weakens consumption smoothing. We use the *full* internal-habit FOC above. If time pressure forces the shortcut, that choice must be labeled “internal habit, no future-habit adjustment” in `dsge/model.py` and in the README, and the log-linearization must be consistent with whichever form is used.

Rule-of-thumb households (fraction λ): no savings, consume all current-period disposable income. Following Galí–López-Salido–Vallés (2007) the budget is

$$(1 + \tau_{C,t}) C_t^{rot} = (1 - \tau_{L,t}) W_t L_t^{rot} + T_t^{rot},$$

so transfers T_t^{rot} add to rule-of-thumb consumption (lump-sum transfers are not Ricardian-equivalent here). The sign convention for T is therefore: $T > 0$ is a transfer from government to households. Any alternative convention (e.g., $T < 0$ as a tax) must be flagged in `dsge/calibration.py`.

Aggregate consumption: $C_t = (1 - \lambda)C_t^o + \lambda C_t^{rot}$.

Field	Decision
Forward-looking share	$1 - \lambda$, slider default TBD in <code>config.py</code>
Habit parameter	$h \in [0, 1)$, slider in sidebar
Utility over consumption	CRRA / log in deviations from habit
Labor disutility	Standard separable; wage Calvo handled at the union level
Budget constraint	Include τ_C , τ_L , τ_K , lump-sum T , bonds B

3.3. Firms and price/wage stickiness

$$\pi_t - \pi_t^{index} = \beta \mathbb{E}_t[\pi_{t+1} - \pi_{t+1}^{index}] + \kappa_p \widehat{mc}_t, \quad \kappa_p = \frac{(1 - \theta_p)(1 - \beta\theta_p)}{\theta_p}.$$

$$\pi_t^w - \pi_t^{w,index} = \beta \mathbb{E}_t[\pi_{t+1}^w - \pi_{t+1}^{w,index}] + \kappa_w (\widehat{mrs}_t - \widehat{w}_t), \quad \kappa_w = \frac{(1 - \theta_w)(1 - \beta\theta_w)}{\theta_w(1 + \varphi\epsilon_w)}.$$

Field	Decision
Price stickiness	Calvo θ_p , slider
Wage stickiness	Calvo θ_w , slider
Capital utilization	Cost convexity ψ , slider; variable u_t with quadratic cost
Production	Cobb–Douglas $Y_t = A_t K_t^\alpha L_t^{1-\alpha}$ in levels; log-linearized
Marginal cost	$\widehat{mc}_t = (1 - \alpha)\widehat{w}_t + \alpha r_t^k - \widehat{A}_t$

Calvo sign trap. When the AI writes the Phillips curve it often drops the intercept in κ_p or mixes indexed and non-indexed versions. Before trusting the output, verify that $\theta_p \rightarrow 0$ gives flexible prices ($\kappa_p \rightarrow \infty$) and $\theta_p \rightarrow 1$ gives fully sticky prices ($\kappa_p \rightarrow 0$). Same logic for θ_w .

3.4. Government and fiscal block

$$B_{t+1} = (1 + r_t) B_t + G_{c,t} + G_{I,t} + T_t - \tau_{C,t} C_t - \tau_{L,t} W_t L_t - \tau_{K,t} r_t^k K_t.$$

Sign conventions: B_t is the real stock of one-period government debt, r_t is the real return on that debt (tied to the policy rate through the Fisher relation in §3.5), $T_t > 0$ is a lump-sum transfer *from government to households* (appears as a use of funds on the left-hand side, consistent with §3.2), and tax revenues enter negatively because they shrink the debt stock. These conventions must be matched exactly in `dsge/model.py`; a sign mistake here will usually produce explosive debt and a silent BK failure.

$$\text{Fiscal rule: } \tau_{X,t} = \rho_X \tau_{X,t-1} + \phi_{b,X} \widehat{B}_t + \varepsilon_{X,t}, \quad X \in \{T, \tau_C, \tau_L, \tau_K, G_c\}.$$

The coefficient $\phi_{b,X}$ is *instrument-specific*. For tax instruments (τ_C, τ_L, τ_K) rising debt calls for higher taxes, so $\phi_{b,X} > 0$. For the G_c -cut rule the government spending *falls* when debt rises, so $\phi_{b,G_c} < 0$. For the lump-sum rule a rise in $\widehat{B}_t$ triggers a *reduction* in transfers (T), again implying a negative coefficient if T is signed as a transfer. The UI slider exposes the magnitude $\phi_b \geq 0$; the sign is applied inside `policy/financing.py` depending on the selected instrument.

Sign-flip trap (G_c -cut and lump-sum rules). A naive implementation applies a single positive ϕ_b to every instrument, which turns the “ G_c cuts when debt rises” rule into “ G_c rises when debt rises” – i.e., the opposite of what the assignment asks for, and explosive. Verify the sign of the debt response numerically: under the G_c -cut rule, a positive debt impulse must produce a decline in $G_{c,t}$.

Field	Decision
Debt feedback	ϕ_b , slider; must be strong enough to stabilize debt
Financing rule	Radio/dropdown picks <i>which</i> instrument reacts to debt: Lump-Sum, τ_C , τ_L , τ_K , or G_c cut
Shock menu	G_c up, G_I up, τ_L cut, τ_K cut
Government debt	Track $\hat{B}_t$ path explicitly for IRF panel

Budget-constraint trap. The AI will often write a fiscal shock without closing the government budget constraint. If debt is not stabilized, the model has no stable saddle path and the QZ solver will either fail or silently return nonsense. Always verify the chosen financing instrument actually moves in response to $\hat{B}_t$.

3.5. Monetary policy block (locked in before Step 5)

The baseline nominal anchor is a standard Taylor rule on inflation and the output gap, with interest-rate smoothing:

$$\hat{i}_t = \rho_i \hat{i}_{t-1} + (1 - \rho_i) [\phi_\pi \hat{\pi}_t + \phi_y \hat{y}_t] + \varepsilon_{i,t},$$

with baseline $\rho_i = 0.8$, $\phi_\pi = 1.5$, $\phi_y = 0.125$. The Fisher relation $\hat{r}_t = \hat{i}_t - \mathbb{E}_t \hat{\pi}_{t+1}$ connects the nominal rate to the real return used in the Euler equation and the government budget.

Non-optional. The audit of an earlier draft flagged monetary policy as “the big hole”: without a rule that pins down $\hat{i}_t$, the two Phillips curves do not close, the system is under-determined, and QZ will either fail BK or silently return indeterminate solutions. The Taylor rule above must be in `dsge/model.py` *before* Step 6 runs, or (only as a fallback) the group must impose an explicit interest-rate peg $\hat{i}_t = 0$ plus a Wicksellian price-level target.

Field	Decision
Rule family	Taylor rule with smoothing on $\hat{i}_t, \hat{\pi}_t, \hat{y}_t$
Baseline coefficients	$\rho_i = 0.8$, $\phi_\pi = 1.5$, $\phi_y = 0.125$; sliders optional, default fixed
Determinacy region	Require $\phi_\pi > 1$ (Taylor principle) – flag a warning in the UI if the user drops below
Zero lower bound	Not modeled; the system is in deviations from steady state

3.6. Shock processes

All structural shocks follow AR(1) processes in logs:

$$\varepsilon_{X,t} = \rho_X \varepsilon_{X,t-1} + \sigma_X u_{X,t}, \quad u_{X,t} \sim \mathcal{N}(0, 1).$$

Structural shocks carried through Tab 1: TFP (A_t), investment-specific, price markup, wage markup, government consumption, **monetary policy** (Taylor-rule residual $\varepsilon_{i,t}$), risk premium. The group can start with a subset, but TFP plus the monetary shock must be present so the model produces non-degenerate output and inflation dynamics. The fiscal shocks in Tab 2 are imposed deterministically.

3.7. Cross-module coding convention

All model-solve functions should return the same top-level structure:

- `steady_state` (dict of levels)
- `A_matrix`, `B_matrix` (state-space transition)
- `C_matrix`, `D_matrix` (observation)
- `variable_index` (name $\rightarrow$ row)
- `shock_index` (name $\rightarrow$ column)
- `diagnostics` (BK conditions, eigenvalues, solver success flag)

4. Step-by-Step Build Plan

4.1. Step 1: Lock the equations before anyone writes serious code

Owner: Shared group work

Open these files first:

- the assignment PDF (`Assignment Final.pdf`),
- the grading rubric (`Grading Rubric.pdf`),
- course notes for Smets–Wouters / DSGE,
- `plan.tex`.

What to do:

- Confirm the household Euler equation (including habit), labor supply, price and wage Phillips curves, capital accumulation with utilization, and the government budget constraint.
- Agree on variable naming: `y_hat`, `c_hat`, `i_hat`, `l_hat`, `pi_hat`, `w_hat`, `b_hat`, `tau_L_hat`, etc.
- Decide exactly which moments Tab 1 will report, in what order.
- Decide which four IRF subplots Tab 2 will show (Output, Consumption, Investment, Debt per the assignment).

Done when: every teammate can answer “what are the state variables, what are the jump variables, what closes the government budget, and what pins down the steady state?”

4.2. Step 2: Make sure the environment and root files are usable

Owner: Shared group work

Create or edit these files:

- `requirements.txt` (streamlit, numpy, scipy, pandas, matplotlib, plotly, pytest)
- `README.md`
- `app.py` (skeleton only at this stage)
- `config.py`

What to do:

- Set up a virtual environment.
- Confirm `scipy.linalg.ordqz` is importable – this is the QZ solver for the rational-expectations system.
- Keep `README.md` short and accurate so any teammate can install and launch the app.
- Use `app.py` as the entry point for a “hello world” smoke test.

- Use `config.py` for default parameters (baseline calibration).

Beginner note: if you are unsure whether something belongs in `app.py` or `config.py`, put only user-interface code in `app.py` and put default structural values in `config.py`.

Done when: `pip install -r requirements.txt` works and `streamlit run app.py` opens a basic page with the two empty tabs wired in.

4.3. Step 3: Create the folder skeleton and lock the git workflow

Owner: Shared group work

Use the structure shown in Section 2.2.

What to do:

- Create the `dsge/`, `solvers/`, `simulation/`, `policy/`, `utils/`, `tests/`, `assets/` folders.
- Add an `__init__.py` file to each so imports work cleanly.
- Commit the skeleton to `main`. Everyone pulls before starting their owned step.

Git workflow (all six teammates). Six people editing one repo without a branching convention is a merge-conflict factory. Lock in: every step gets a feature branch (`step-5a-private-block`, `step-8b-fred-empirical`, etc.); no direct pushes to `main`; every merge goes through a pull request with at least one teammate review; the branch must pass `pytest` before merge. Large refactors (anything touching `dsge/model.py` after Step 5b) should announce on the group chat first so parallel work does not collide.

Done when: the group has agreed where each type of code will live, the branching convention is written into `README.md`, and everyone has pulled the skeleton before the real implementation starts.

4.4. Step 4: Calibrate the model and build the steady state

Owner: _____

Open or edit these files:

- `dsge/calibration.py`
- `dsge/steady_state.py`
- `tests/test_steady_state.py`

What to do:

- Set baseline values: $\beta = 0.99$, $\sigma = 1$ (log), $\alpha = 0.33$, $\delta = 0.025$, ϵ_p and ϵ_w elasticities, G/Y target, debt/GDP target, average tax rates, shock persistences and standard deviations.
- Solve the deterministic steady state. For Smets–Wouters this is mostly analytical: pick r^k from β , back out K/L , then w , Y , I , C from ratios.
- Return a dictionary of steady-state levels used by every other module.

Done when: the steady state satisfies the resource constraint, the government budget, and the labor first-order condition to within numerical tolerance, and `test_steady_state.py` passes.

4.5. Step 5a: Linearize the private-sector block (households and firms)

Owner: _____

Open or edit these files:

- `dsge/model.py` (household + firm equations)
- `tests/test_model_private.py`

What to do:

- Log-linearize: Euler equation with habit (full FOC from §3.2), rule-of-thumb consumption, aggregate consumption, capital accumulation with utilization, investment first-order condition (Q equation), labor supply, price Phillips curve, wage Phillips curve, production function, marginal cost, resource constraint.
- Watch the timing convention: capital entering period- t production is K_{t-1} (end-of-previous-period stock). This is a common AI-hallucination trap.
- Unit-test by plugging in arbitrary coefficients and checking matrix shapes; test that $\kappa_p \rightarrow 0$ as $\theta_p \rightarrow 1$.

Done when: the private-block equations compile into the row-block of Γ_0, Γ_1 with the correct dimensions and pass shape / sign tests.

4.6. Step 5b: Linearize the policy block and build the full state-space form

Owner: _____ (different person from Step 5a)

Open or edit these files:

- `dsge/model.py` (government + monetary equations)
- `solvers/state_space.py`
- `tests/test_model_policy.py`

What to do:

- Log-linearize: government budget constraint, fiscal rule (with the instrument-specific sign from §3.4), Taylor rule, Fisher relation, AR(1)s for every structural shock including the Taylor-rule residual.
- Assemble the full Sims/Klein canonical form

$$\Gamma_0 x_t = \Gamma_1 x_{t-1} + \Psi \varepsilon_t + \Pi \eta_t,$$

where η_t collects expectational errors.

- Expose `build_system(params)` returning $\Gamma_0, \Gamma_1, \Psi, \Pi$ and a variable/shock index.

License for downstream owners to start early. Owners of Steps 6–11 should *not* wait for Step 5b to finish. Each downstream module takes a dict with keys `A_matrix`, `B_matrix`, `variable_index`, `shock_index` (see §3.6). While Step 5 is in progress, Steps 6–11 can be written against a stub that returns a small, hand-coded (A, B) for a toy two-equation system. The integration test comes at the end of Step 5b.

Done when: `build_system(params)` returns $\Gamma_0, \Gamma_1, \Psi, \Pi$ with correct dimensions, no hard-coded magic numbers, and private-block + policy-block integration tests both pass.

4.7. Step 6: Solve the rational-expectations system with QZ decomposition

Owner: _____

Open or edit these files:

- `solvers/rational_expectations.py`
- `tests/test_blanchard_kahn.py`

What to do:

- Use `scipy.linalg.ordqz` to order the generalized Schur decomposition.
- Count explosive eigenvalues $|\lambda| > 1 + \varepsilon_{\text{tol}}$ and compare to the number of jump variables – Blanchard–Kahn condition. Use a documented numerical tolerance (default $\varepsilon_{\text{tol}} = 10^{-6}$) so that a near-unit-root eigenvalue does not flip the BK count unpredictably.

- Handle complex conjugate pairs: count the pair together, not as two separate explosive roots, and log any eigenvalues that fall inside a small band around $|\lambda| = 1$.
- Return the reduced-form transition $x_{t+1} = Ax_t + B\varepsilon_{t+1}$ plus a `converged` / `indeterminacy` / `no_solution` flag, along with the eigenvalue list for diagnostics.
- If `ordqz` raises or the reordering pivot is ill-conditioned, catch the exception, tag the result as `no_solution`, and return rather than crashing the app.
- Use the same callback pattern as Project 2's solver so downstream code does not care which algorithm was chosen.

```
AA, BB, alpha, beta_, Q, Z = scipy.linalg.ordqz(
    Gamma0, Gamma1, sort=lambda a, b: np.abs(a) < np.abs(b)
)
# Blanchard-Kahn: # explosive eigenvalues == # jump variables
```

Done when: for a baseline calibration the solver reports BK satisfied, and for a deliberately bad calibration (e.g., $\phi_b = 0$) it reports indeterminacy or no solution instead of silently returning garbage.

4.8. Step 7: Add sanity checks for eigenvalues and budget constraints

Owner: _____

Open or edit these files:

- `solvers/rational_expectations.py`
- `dsge/model.py`
- `tests/test_blanchard_kahn.py`

What to do:

- Add a pre-solve check that every equation in `dsge/model.py` is in the system exactly once.
- Add a post-solve check that the government budget residual, computed from the simulated paths, is zero up to numerical tolerance.
- For Tab 2, verify that the chosen financing rule actually moves when $\hat{B}_t$ moves (and in the *correct direction* – see §3.4 sign-flip warning).
- **Indeterminacy UX.** When the solver flag is `indeterminacy` or `no_solution`, surface a `st.error` at the top of Tab 1 and Tab 2 with a one-line suggestion based on which parameter is most likely responsible (e.g., ϕ_b too small, $\phi_\pi < 1$, θ_p at a boundary). Blank the IRF plots rather than rendering a stale cached figure.
- **Separate caching.** Cache the *solve* (structural parameters $\rightarrow (A, B)$) and the *simulate* (shock variances, seed $\rightarrow$ simulated moments) on different `@st.cache_data` decorators. Moving a shock-variance slider should not trigger a re-solve; moving a Calvo slider should invalidate the solve but keep the RNG seed stable.

Done when: extreme parameter choices fail cleanly (with a Streamlit-visible message) instead of producing nonsense IRFs, and the app does not re-solve when only shock-variance sliders move.

4.9. Step 8: Build Tab 1 – unconditional simulation and moments

Owner: _____

Open or edit these files:

- `simulation/simulate.py`
- `simulation/moments.py`
- `tests/test_moments.py`

What to do:

- Given A and B , draw $T = 1000$ periods of i.i.d. normal shocks and propagate $x_{t+1} = Ax_t + B\varepsilon_{t+1}$.
- Extract observable series: $\hat{y}_t, \hat{c}_t, \hat{i}_t, \hat{l}_t, \hat{\pi}_t$.
- Compute at least 10 moments: variance of each, correlation of each with output, and first-order autocorrelation of each.
- **Filtering convention.** The model produces log-deviations from steady state directly, so no detrending is needed on the *simulated* side. When comparing to the FRED empirical block in Step 8b, apply an HP filter ($\lambda = 1600$, quarterly) to the empirical series so the two sides are on the same footing. Store both the raw log-deviation moments and the HP-filtered moments.
- Seed the RNG so the same parameters give the same numbers.

Done when: the same seed gives identical moments, and the variance ratios pass basic sanity (consumption volatility should fall below output volatility when h is high; investment volatility should exceed output volatility).

4.10. Step 8b: Empirical moments from FRED (Data Retrieval rubric row)

Owner: _____ (pairs with Step 8 owner)

Open or edit these files:

- simulation/empirical.py
- tests/test_empirical.py

What to do:

- Pull US quarterly real GDP (GDPC1), real personal consumption (PCECC96), real private nonresidential investment (PNFIC1), aggregate hours (e.g., HOABS), and CPI inflation (CPIAUCSL) from FRED via `pandas-datareader` or `fredapi`.
- Log-transform levels, HP-filter with $\lambda = 1600$, and compute the same moments (variance, output correlation, first-order autocorrelation) on the cyclical components.
- In Tab 1, render a side-by-side table: model moments vs. empirical moments, with a ratio or absolute-difference column. The f-string commentary in Step 9 should reference this comparison (“the model reproduces X/Y of empirical consumption volatility”).
- Cache the FRED pull on disk (`@st.cache_data` with `ttl=24h`); the data should not be refetched on every slider move.

Why this step exists. The rubric has a Data Retrieval & Integrity row that a pure theory model does not naturally satisfy. Adding an empirical moment-matching block turns moment comparison into a genuine data exercise (retrieve, clean, align frequency, filter, document) and is what lifts that rubric row from 3 to 4.

Done when: the empirical moment table loads on first app launch, Tab 1 shows model and empirical moments next to each other, the FRED fetch is cached, and the difference column updates whenever the structural sliders move.

4.11. Step 9: Build Tab 1 dynamic commentary

Owner: _____

Open or edit these files:

- utils/summaries.py
- app.py (Tab 1 section)

What to do:

- Write deterministic f-string generators that take the moments dictionary and output a paragraph or two of plain-English interpretation.

- Examples: “With $h = 0.7$, consumption volatility is $X\%$ of output volatility, consistent with the stylized fact that consumption is smoother than output.”
- Tie at least one sentence to each slider: habit, utilization, price stickiness, wage stickiness, debt feedback.

Done when: the text block in Tab 1 changes correctly and non-trivially as each slider moves.

4.12. Step 10: Build Tab 2 – fiscal shocks and financing rules

Owner: _____

Open or edit these files:

- policy/shocks.py
- policy/financing.py
- simulation/irf.py

What to do:

- Define each fiscal shock as a unit impulse to the appropriate AR(1) innovation: ε_{G_c} , ε_{G_I} , negative ε_{τ_L} , negative ε_{τ_K} .
- Implement the financing-rule selector: modify which fiscal instrument has non-zero ϕ_b coefficient before calling the solver.
- Compute the 40-quarter IRF by setting $x_0 = 0$, injecting the chosen ε , and iterating $x_{t+1} = Ax_t$.
- Return IRFs for at least output, consumption, investment, and government debt; add extras (inflation, labor, real wage) if easy.

Done when: switching the financing rule from Lump-Sum to Capital Tax Hikes visibly reduces the cumulative output response (the Q&A question in the assignment).

4.13. Step 11: Multipliers and Fiscal Drag Horizon

Owner: _____

Open or edit these files:

- policy/multipliers.py
- tests/test_multipliers.py

What to do:

- Impact multiplier: $IM = \Delta Y_0 / \Delta G_0$ in level terms (convert from log deviations using steady-state ratios).
- Cumulative multiplier: $CM(H) = \frac{\sum_{t=0}^H \beta^t \Delta Y_t}{\sum_{t=0}^H \beta^t \Delta G_t}$.
- Fiscal Drag Horizon: smallest $t^* \geq 1$ with $\Delta Y_{t^*} < 0$ (return **None** if it never crosses within the 40-quarter horizon).
- Provide both metrics to the policy-briefing f-string.

Done when: under Lump-Sum financing the multipliers match standard textbook magnitudes for a medium-scale DSGE (IM roughly in $[0.3, 1.2]$ depending on rule-of-thumb share), and distortionary financing visibly lowers them.

4.14. Step 11b: Econometric analysis on simulated data (Econometrics rubric row)

Owner: _____ (pairs with Step 11 owner)

Open or edit these files:

- simulation/econometrics.py
- tests/test_econometrics.py

What to do:

- On the 1,000-period simulated series, run at least two OLS regressions with reported coefficients, standard errors, and R^2 :
 - **Taylor-rule recovery:** regress $\hat{i}_t$ on $\hat{\pi}_t, \hat{y}_t, \hat{i}_{t-1}$ and recover $\hat{\phi}_\pi, \hat{\phi}_y, \hat{\rho}_i$. Compare to the structural values set in §3.5.
 - **Consumption smoothness:** regress $\hat{c}_t$ on $\hat{y}_t$ (plus lags) to recover a smoothness coefficient; compare across habit settings.
- Report variance ratios ($\text{Var}(\hat{c})/\text{Var}(\hat{y})$, etc.) with bootstrapped standard errors.
- Display the regression table in Tab 1 as a small panel below the moments table.

Why this step exists. The Econometric Analysis rubric row asks for coefficients, standard errors, and R^2 . A pure DSGE simulation does not produce these unless we explicitly run regressions on the simulated output. This step plugs the gap.

Done when: the Taylor-rule recovery regression produces estimates within a reasonable band of the structural parameters, and the regression panel updates when sliders move.

4.15. Step 12: Build Tab 2 IRF plots and policy briefing

Owner: _____

Open or edit these files:

- `app.py` (Tab 2 section)
- `utils/summaries.py`

What to do:

- Plot four IRF subplots (Output, Consumption, Investment, Debt) with labeled axes, zero-line, and a vertical marker at the Fiscal Drag Horizon.
- **Composite chart (Visualization rubric row).** Add at least one composite view that shows multiple financing rules simultaneously. The chosen baseline is an overlay of the Output IRF across all five financing rules on the same axes (lump-sum, τ_C , τ_L , τ_K , G_c -cut), with a legend and a shaded band showing the range of cumulative multipliers. A second, optional composite: a heatmap of cumulative multiplier as a function of ϕ_b (y-axis) $\times$ financing rule (x-axis).
- Add the f-string policy briefing: what shock was run, which financing rule was selected, impact and cumulative multipliers, transmission mechanism summary, drag horizon, and one sentence comparing the selected rule to the lump-sum benchmark on the composite chart.

Done when: a user can click through each shock $\times$ financing-rule combination and get both a coherent picture and coherent text; the composite overlay makes multiplier dispersion visually obvious without the user having to toggle rules one at a time.

4.16. Step 13: Global sidebar and parameter plumbing

Owner: Shared group work

Open or edit these files:

- `app.py`
- `config.py`

What to do:

- Add continuous `st.slider` controls for h , ψ , θ_p , θ_w , ϕ_b , rule-of-thumb share λ , and shock persistences/standard deviations.
- Use the locked-in slider ranges in the table below so boundary cases never put Calvo or habit parameters at degenerate values.

- Cache the *solve* and the *simulate* on separate `@st.cache_data` calls so that moving a shock-variance slider (which only changes the simulation) does not force a re-solve, and moving a structural slider (which changes A, B) does not invalidate unrelated UI state.
- Confirm the two tabs share a single parameter block via `st.session_state`.
- If the solver returns `indeterminacy` or `no_solution`, render the dashboard with `st.error()` and a one-line suggestion (e.g., “raise ϕ_b above 0.02” or “keep $\phi_\pi > 1$ ”). Never let the IRF plots render on silent failure.

Parameter	Slider range	Notes
h	[0.00, 0.95]	Closed below, open above 1. Default 0.70.
ψ	[0.01, 5.00]	Cost-of-utilization convexity. 0 = free utilization (degenerate); cap at 5 to avoid numerical overflow. Default 0.50.
θ_p	[0.10, 0.95]	Calvo price. Open bounds avoid $\kappa_p \rightarrow \infty$ (perfectly flexible) and $\kappa_p = 0$ (perfectly sticky). Default 0.75.
θ_w	[0.10, 0.95]	Calvo wage. Same logic. Default 0.75.
ϕ_b	[0.00, 0.20]	Debt-feedback intensity (magnitude). Values below ≈ 0.01 typically fail BK. Default 0.05.
λ	[0.00, 0.60]	Rule-of-thumb share. Default 0.30. Above ≈ 0.6 multipliers become unrealistic.
ρ_X	[0.00, 0.99]	AR(1) persistence. Default 0.85.
σ_X	[0.001, 0.05]	Shock sd. Default 0.007.

Done when: a user can move any structural slider and both tabs update within a few seconds, and every parameter slider has explicit, labeled bounds.

4.17. Step 14: Styling and polish

Owner: Shared group work

Open or edit these files:

- `assets/style.css`
- `app.py`

What to do:

- Dark/navy background (`#0e1a2b` or similar), bright white chart cards, high-contrast text.
- Import Garamond (or EB Garamond) with a serif fallback so the app degrades gracefully.
- Ensure tab labels, widget labels, and axis labels are readable on both background states.
- Use consistent colors: one color for output, one for consumption, one for investment, one for debt.

Done when: the app looks intentional; any teammate would be happy to open it in front of the professor.

4.18. Step 15: Testing and sanity suite (written *alongside* each step, not after)

Owner: Shared group work

Test-driven discipline. Do not schedule the entire test suite for the end of the build. Each step owner writes the test file for their step *at the same time as the code*. Step 15 is the aggregation / polish pass, not the first time tests get written. The filenames below map one-to-one onto the code steps above (4, 5a, 5b, 6–7, 8, 8b, 10–11, 11b), so a missing test file is a missing step.

Create or edit these files inside the existing `tests/` folder:

- `tests/test_steady_state.py`

- tests/test_model_private.py
- tests/test_model_policy.py
- tests/test_blanchard_kahn.py
- tests/test_moments.py
- tests/test_empirical.py
- tests/test_irf.py
- tests/test_multipliers.py
- tests/test_econometrics.py

What to do:

- Test that the steady state satisfies every aggregate identity (resource constraint, government budget, labor FOC).
- Test that BK conditions hold at baseline and fail under each deliberately bad calibration (e.g., $\phi_b = 0$, $\phi_\pi < 1$, $\theta_p \rightarrow 1^-$).
- Test that moments match direct `numpy` computations on the simulated series, for both raw and HP-filtered inputs.
- **IRF tests (beefed up).** “Decays to zero” is not enough on its own. Also check: the *sign of the peak* response for each shock (e.g., a G_c shock has positive peak output), the quarter of the peak (should be 0–4 for most shocks), and that under lump-sum financing the impact multiplier lies inside a target band derived from an analytical stripped-down RBC case.
- Test the sign of the debt response under each financing rule, in particular that under the G_c -cut rule a positive debt impulse generates a *negative* G_c response.
- Test multipliers under lump-sum financing against a hand-calculated benchmark for the stripped-down RBC case.
- Check for NaNs, infinities, and obviously explosive paths on all IRFs.
- Test the Taylor-rule recovery regression in Step 11b: estimated $\hat{\phi}_\pi$ should land within 10–15% of the structural value when the sample is long enough.

Done when: the sanity suite passes on a fresh clone, each step has its paired test file, and the dashboard outputs match the underlying numerical data.

4.19. Step 16: Prompt log and comments

Owner: Shared group work

Create or edit:

- prompt_log.md in the project root
- comments inside every non-trivial Python file

What to do:

- For each major AI-assisted component, record the prompt, the file it affected, what was kept, and what was corrected. The assignment explicitly says if the AI misspecifies Calvo, the government budget, or an Euler equation, you must catch and fix it using AI (not by hand).
- Add comments that explain the economics and numerical logic, not just the syntax.

Done when: if the professor points at a block of code during Q&A, someone can explain what it does and how the AI-generated part was checked.

4.20. Step 17: Rehearse the live demo

Owner: Shared group work

What to practice:

- run `pip install -r requirements.txt`
- run `streamlit run app.py`

- show Tab 1 and read out a few moments, explain what the habit slider did to consumption volatility
- switch to Tab 2, run a G_c shock under Lump-Sum financing, then switch to Capital Tax Hikes, and explain why the cumulative multiplier dropped
- answer code-location questions such as “where is the QZ decomposition?”, “where is the government budget constraint enforced?”, “where is the rule-of-thumb aggregation?”

Expected Q&A from the assignment PDF. Be ready for: “Explain how your prompt incorporated rule-of-thumb consumers into the aggregate consumption Euler equation,” and “Why does shifting the financing rule from Lump-Sum to Capital Tax Hikes crash the cumulative multiplier?” Every teammate should have a two-sentence answer to both.

Done when: any teammate can handle both a random economics question and a random code-workflow question without needing a rescue, in under 10 minutes total.

5. Team Ownership By Step

Step	Owner	Main responsibility
1–3	Shared	Lock equations, confirm environment, build folder skeleton, agree on git workflow
4	_____	Calibration and steady state
5a	_____	Linearized private-sector block (households + firms)
5b	_____	Linearized policy block + full state-space assembly
6–7	_____	QZ decomposition solver and BK / budget-constraint checks
8–9	_____	Tab 1: unconditional simulation, moments, dynamic commentary
8b	_____	FRED empirical moments, HP filter, model-vs-data table
10–11	_____	Tab 2: fiscal shocks, financing rules, multipliers, Fiscal Drag Horizon
11b	_____	Taylor-rule / consumption-smoothness regressions on simulated data
12	_____	Tab 2 IRF plots, composite overlay, automated policy briefing
13–17	Shared	Sidebar plumbing, styling, tests, prompt log, rehearsal

How to use this ownership table. It does not mean one person works alone forever. It means that person is the first owner of the step, writes the first clean version, and makes sure the rest of the group understands it before the team moves on. Fill in names at the start of the project – before anyone starts typing Python.

5.1. Where each member’s work lives in the final project

Use this box during the presentation to quickly point at the file that contains your contribution. If the professor asks “who wrote this and where is it?” every teammate should be able to answer immediately. Fill this in as work lands.

– **Step 4: Calibration and steady state.**

- `dsge/calibration.py` – baseline parameter dict.
- `dsge/steady_state.py` – analytical SS solver returning levels.
- `tests/test_steady_state.py` – resource-constraint, budget-constraint, and FOC residual checks.

– **Step 5: Linearized equations and state-space form.**

- `dsge/model.py` – Euler w/ habit, rule-of-thumb, Phillips curves, wage Phillips curve, capital accumulation w/ utilization, Q equation, resource constraint, government budget, fiscal rule, shock AR(1)s.
- `solvers/state_space.py` – builds $\Gamma_0, \Gamma_1, \Psi, \Pi$ from the parameter dict.

– **Steps 6–7: QZ solver and sanity checks.**

- `solvers/rational_expectations.py` – `scipy.linalg.ordqz`, BK count, returns A, B .
- `tests/test_blanchard_kahn.py` – baseline satisfies BK; bad calibration fails cleanly.
- Post-solve budget-constraint residual check integrated into `dsge/model.py`.

– **Steps 8–9: Tab 1 simulation and commentary.**

- `simulation/simulate.py` – 1,000-period stochastic simulation.
- `simulation/moments.py` – ≥ 10 moments (variances, output correlations, first-order autocorrelations).
- `utils/summaries.py` – Tab 1 f-string commentary keyed to each slider.
- `tests/test_moments.py` – moments match direct numpy calls.

– **Steps 10–11: Fiscal shocks, financing rules, multipliers.**

- `policy/shocks.py` – G_c, G_I, τ_L cut, τ_K cut as unit impulses.
- `policy/financing.py` – Lump-Sum, $\tau_C, \tau_L, \tau_K, G_c$ -cut rules.
- `policy/multipliers.py` – Impact and cumulative (discounted) multipliers, Fiscal Drag Horizon.
- `simulation/irf.py` – 40-quarter deterministic response.
- `tests/test_multipliers.py` – RBC-case benchmark.

– **Step 12: Tab 2 visuals and briefing.**

- `app.py` Tab 2 – 4 IRF subplots, drag-horizon vertical marker, impact/cumulative multiplier readout.
- `utils/summaries.py` – policy briefing f-string (shock + financing-rule interaction + drag horizon).

Shared – Steps 13–17: Sidebar, styling, tests, prompt log, rehearsal.

- `app.py` sidebar: continuous sliders for $h, \psi, \theta_p, \theta_w, \phi_b, \lambda$, shock persistences.
- `app.py` caching: `@st.cache_data` on solve step.
- `config.py` – defaults; `assets/style.css` – dark/navy + Garamond.
- Full tests/ suite, `prompt_log.md`, and live-demo rehearsal.

6. Final Demo and Validation Checklist

6.1. Definition of done

1. The linearized equations match the course notes and Smets–Wouters structure.
2. The steady state satisfies every aggregate identity at machine precision.

3. The QZ solver satisfies Blanchard–Kahn at the baseline and fails cleanly otherwise.
4. Tab 1 reports at least 10 business-cycle moments over 1,000 periods.
5. Tab 1 commentary responds to every sidebar slider.
6. Tab 2 supports G_c , G_I , τ_L cut, and τ_K cut shocks.
7. Tab 2 supports at least Lump-Sum, τ_C , τ_L , τ_K , and G_c -cut financing rules.
8. Tab 2 reports both impact and cumulative multipliers plus the Fiscal Drag Horizon.
9. The dashboard uses dark/navy styling, white charting areas, and Garamond.
10. The code is commented and `prompt_log.md` exists.
11. The project runs from scratch on the laptop used for the demo.

6.2. Live-demo checklist

- `pip install -r requirements.txt` works cleanly.
- `streamlit run app.py` launches without manual fixes.
- Both tabs load; the sidebar sliders update both tabs in real time.
- Tab 1 moments are numerically correct.
- Tab 2 IRF subplots for Output, Consumption, Investment, Debt are labeled and readable.
- Switching the financing rule visibly changes the cumulative multiplier.
- The team can explain: habit formation, Calvo pricing, the government budget constraint, the rule-of-thumb Euler aggregation, QZ / Blanchard–Kahn, impact vs. cumulative multiplier, and the Fiscal Drag Horizon.

Bottom line. The best version of this assignment is the one where the economics are locked in first, the numerical methods (steady state, QZ, BK) are trustworthy second, and the dashboard polish comes third. If the group can launch the app from scratch, toggle shocks and financing rules live, and explain exactly why the multipliers move the way they do, the project is in strong shape.